

数据科学导论 第四次作业

厉帅 10235501418

作业内容：

完成“数据科学与工程导论-06”课后题的复习题的第1-6题;练习题7、8题。

复习题

1. 数据管理的生命周期涵盖从数据的生成到最终的销毁。最初，数据通过各种方式被生成和采集，然后存储在数据库或数据仓库中。接着，数据必须被处理和清洗，以确保其质量和格式适合进一步分析。数据分析阶段通过使用各种分析技术和工具提取有用的信息。当数据分析产生的见解需要共享时，数据可以通过适当的渠道被分发。最终，数据可能需要归档以便长期保留，或者在不再需要时被安全销毁。
2. 数据采集是指收集和获取数据以便后续处理和分析的过程。采集的方法多种多样，包括自动化采集，如通过传感器收集的数据，日志文件和API调用；手动采集，涉及问卷调查和访谈等人力方法；网络抓取，则通过爬虫技术从网上获取数据；还有数据导入，从其他数据库或文件系统中导入数据。
3. 数据管理涵盖了数据的各个方面，从获取、存储到保护和处理。在比较传统的数据管理与大数据管理时，传统管理更依赖于关系型数据库和数据仓库，而且处理的数据量较小；而大数据管理则需要应对海量、多样的数据，强调实时处理，因此使用分布式系统和技术，如Hadoop、Spark，以及NoSQL数据库。两者虽然都关注数据质量和安全，但大数据技术在处理速度和可扩展性上有显著优势。
4. 大数据计算可以根据应用场景分为几种模式。批处理模式，比如MapReduce，用于处理大量数据时的高效计算。流处理模式，如Apache Kafka和Storm，被用来处理实时数据流。交互式分析模式，使用如Hive或Presto，允许用户方便地进行数据查询和分析。最后，还有图计算模式，用于处理和分析庞大的图形数据结构，常见框架包括GraphX和Neo4j。
5. 数据分析是从数据中提取见解和支持决策的过程。分析的方法多样，包括描述性分析（总结数据特征）、诊断性分析（理解数据背后的原因）、预测性分析（预测未来趋势）、和规范性分析（建议行动方案）。此外，数据分析常借助于机器学习模型，如回归分析、聚类分析和分类方法。
6. 将数据转化为可视化的图形或图表，有助于更好地理解 and 解释数据。它提高了数据的可读性，使决策者更容易识别趋势和模式。此外，数据可视化可以使复杂的信息更容易传达，特别是对于非技术受众，也提高了数据分析结果的吸引力和沟通效果。可视化能够加深对数据的理解，从而支持更明智的决策。

练习题

7.

```
import numpy as np
import matplotlib.pyplot as plt
```

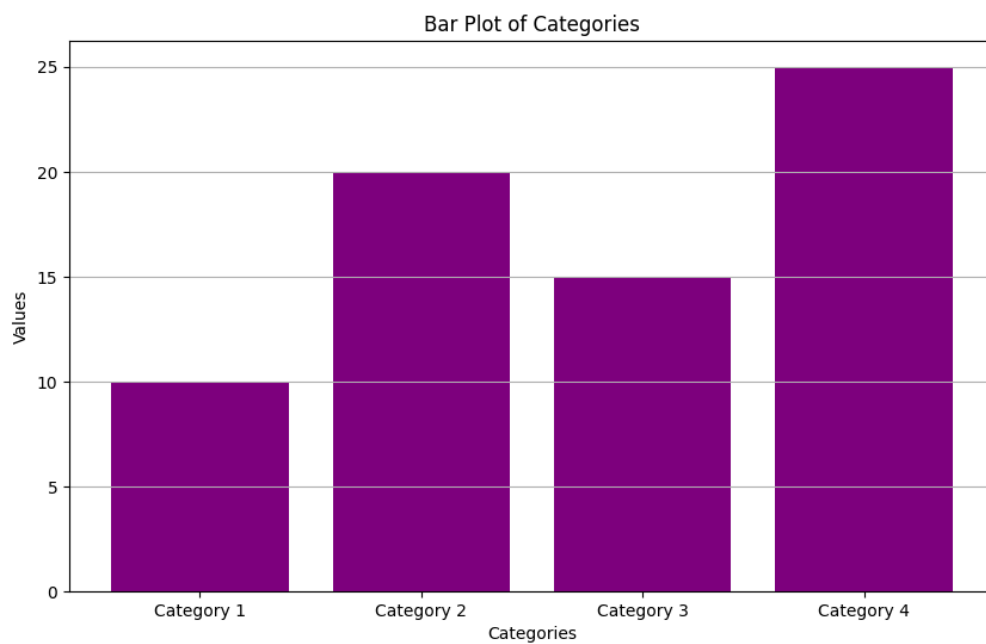
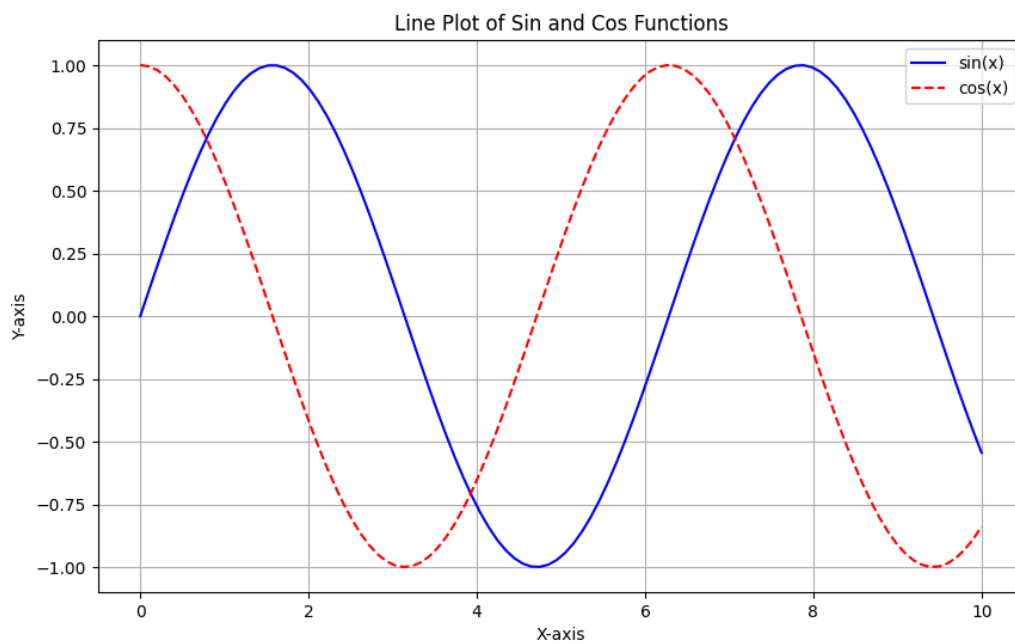
```

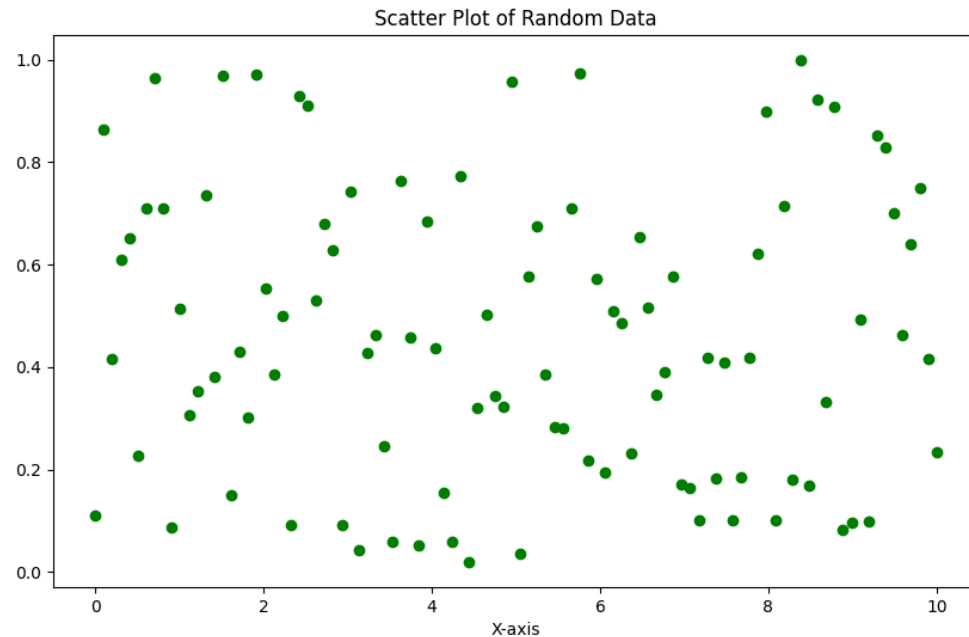
# 创建示例数据
x = np.linspace(0, 10, 100) # x 值从 0 到 10, 分成 100 个点
y1 = np.sin(x)
# y1 为 sin(x)
y2 = np.cos(x)
y3 = np.random.rand(100)
# y2 为 cos(x)
# y3 为随机数据

plt.figure(figsize=(10, 6)) # 设置图形大小
plt.plot(x, y1, label='sin(x)', color='b', linestyle='--') # 绘制 sin(x) 折线
plt.plot(x, y2, label='cos(x)', color='r', linestyle='--') # 绘制 cos(x) 折线
plt.title('Line Plot of Sin and Cos Functions') # 图形标题
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(10, 6)) # 设置图形大小
plt.scatter(x, y3, color='g', marker='o', label='Random Data') # 绘制随机数据的散点
plt.title('Scatter Plot of Random Data') # 图形标题
plt.xlabel('X-axis')
categories = ['Category 1', 'Category 2', 'Category 3', 'Category 4']
values = [10, 20, 15, 25]
plt.figure(figsize=(10, 6)) # 设置图形大小
plt.bar(categories, values, color='purple') # 绘制柱状图
plt.title('Bar Plot of Categories') # 图形标题
plt.xlabel('Categories')
plt.ylabel('Values')
plt.grid(axis='y')
plt.show()

```





8.

```
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
# 创建示例数据
np.random.seed(0) # 设置随机种子以便结果可重现
x = np.linspace(0, 10, 100) # x 值从 0 到 10, 分成 100 个点
y1 = np.sin(x) + np.random.normal(0, 0.1, size=x.shape) # y1 为 sin(x) 加上随机噪
y2 = np.cos(x) + np.random.normal(0, 0.1, size=x.shape) # y2 为 cos(x) 加上随机噪

plt.figure(figsize=(10, 6)) # 设置图形大小
sns.lineplot(x=x, y=y1, label='sin(x)', color='b') # 绘制 sin(x) 折线
sns.lineplot(x=x, y=y2, label='cos(x)', color='r') # 绘制 cos(x) 折线
plt.title('Line Plot of Sin and Cos Functions') # 图形标题
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid()
plt.show()

x_scatter = np.random.rand(100) * 10 # 生成 100 个随机 x 值
y_scatter = np.random.rand(100) * 10 # 生成 100 个随机 y 值
plt.figure(figsize=(10, 6)) # 设置图形大小
sns.scatterplot(x=x_scatter, y=y_scatter, color='g', label='Random Data') # 绘制
plt.title('Scatter Plot of Random Data') # 图形标题
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid()
plt.show()

categories = ['Category 1', 'Category 2', 'Category 3', 'Category 4']
```

```
values = [10, 20, 15, 25]
plt.figure(figsize=(10, 6)) # 设置图形大小
sns.barplot(x=categories, y=values, palette='viridis') # 绘制柱状图
plt.title('Bar Plot of Categories') # 图形标题
plt.xlabel('Categories')
plt.ylabel('Values')
plt.grid(axis='y')
plt.show()
```

